Лекція №11 (2 години). Обробка колекцій даних (Методи масивів).

План лекції

1. Імперативний vs Декларативний підхід у програмуванні.
2. Callback-функції при роботі з масивами.
3. Перебір елементів: метод `forEach`.
4. Трансформація масивів: метод `map`.
5. Фільтрація масивів: метод `filter`.
6. Агрегація даних (Згортання): метод `reduce`.
7. Пошук та сортування: `find`, `findIndex`, `some`, `every`, `sort`.
8. Ланцюжки методів (Chaining).

Перелік умовних скорочень

- **JS** — JavaScript.
- **ES6** (ECMAScript 2015) — ключовий стандарт JavaScript.
- **Callback** — функція зворотного виклику.
- **API** (Application Programming Interface) — інтерфейс прикладного програмування.

Вступ

Більшість часу фронтенд-розробник працює зі списками даних, які отримує від сервера (бек-енду). Це може бути список товарів в інтернет-магазині, список повідомлень у чаті або список транзакцій у банківському додатку. У JavaScript такі списки завжди представлені у вигляді **Масивів Об'єктів** (Array of Objects).

До виходу стандарту ES6 для перебору масивів використовували класичний цикл `for`. Цей підхід змушував писати багато зайвого коду і вручну керувати лічильниками (`let i = 0`). Сучасний JavaScript надає розробникам потужний набір декларативних методів (наприклад, `map`, `filter`, `reduce`), які дозволяють виконувати складні операції з даними в один або два рядки коду, роблячи його чистішим і набагато читабельнішим.

Саме глибоке розуміння та вільне володіння цими методами відрізняє початківця від професійного розробника (Junior vs Middle). Ці методи також критично важливі для роботи з сучасними фреймворками, такими як React (де метод `map` — це основний спосіб відмальовки списків HTML на сторінці).

1. Імперативний та Декларативний підхід

Імперативний підхід (Як зробити?): Ви описуєте комп'ютеру кожен крок, розжовуючи деталі. Класичний цикл `for` — це імперативщина. Ви повинні самостійно створити лічильник `i`, самостійно перевіряти, чи досяг він кінця масиву, і самостійно його збільшувати.

Декларативний підхід (Що я хочу отримати?): Ви лише вказуєте комп'ютеру кінцевий результат, який вас цікавить, за допомогою готових методів. Усі "брудні" деталі (лічильники, перевірки довжини) заховані всередині цих методів і працюють автоматично. Сучасні методи масивів в JS (`map`, `filter`, `reduce`) — це яскравий приклад декларативного підходу.

2. Відродження Callbacks

Усі перелічені нижче методи працюють за одним принципом: вони приймають **Callback-функцію** як свій єдиний аргумент. Метод самостійно "пробігається" по всьому масиву від першого до останнього елемента і, на кожному кроці, **викликає** вашу Callback-функцію, передаючи їй поточний елемент, його індекс та, власне, сам масив.

```
const numbers = [10, 20, 30];

// Синтаксис Callback в методах (зазвичай використовують стрілочні функції)
numbers.someMethod((element, index, array) => {
  // Тіло вашої функції: що робити з кожним element?
});
```

> ***Важливо:*** *У більшості випадків нам потрібен лише перший параметр (`element`), а `index` та `array` ми просто опускаємо (не пишемо).*

3. Перебір елементів: `forEach`

Метод `forEach` ("для кожного") — це пряма, сучасна і більш зручна альтернатива циклу `for`. Він просто бере кожен елемент масиву і щось з ним робить (наприклад, виводить у консоль або змінює зовнішню змінну).

Ключова особливість: `forEach` завжди повертає `undefined`. Він нічого *не створює*, він лише *виконує дію*. Метод `forEach` **не можна** перервати `break` або `continue` (як це можна зробити у звичайному `for`).

```
const students = ["Іван", "Петро", "Марія"];

// Виведе кожне ім'я в консоль
students.forEach((student) => {
  console.log(`Студент: ${student}`);
});
```

4. Трансформація масивів: `map`

Метод `map` ("відображення" / "карта") — мабуть, найважливіший метод у сучасному вебі. Він використовується, коли ми маємо один масив, і хочемо створити з нього **новий масив**, де кожен елемент буде якось модифіковано (трансформовано).

Ключові особливості `map`:

1. Він НЕ змінює оригінальний масив.
2. Він **повертає новий масив** такої ж довжини.
3. Ваш Callback **зобов'язаний повернути (`return`)** якесь значення. Те, що повертає Callback для поточного елемента, записується у новий масив.

```
const prices = [100, 250, 400];
const discount = 0.9; // Знижка 10%

// Створюємо новий масив discountedPrices
const discountedPrices = prices.map((price) => price * discount);

console.log(prices); // [100, 250, 400] (Оригінал не змінився)
console.log(discountedPrices); // [90, 225, 360] (Новий масив)
```

У React метод `map` використовується для трансформації масиву даних (об'єктів) у візуальні елементи JSX (наприклад, у теги `<li>` чи `<Card>`).

5. Фільтрація: `filter`

Як зрозуміло з назви, `filter` потрібен для відсіювання елементів. Ви передаєте йому Callback-функцію з перевіркою (умовою). Якщо умова повертає `true` — елемент проходить фільтр і потрапляє у новий масив. Якщо `false` — елемент ігнорується.

Ключові особливості `filter`:

1. Не змінює оригінал, створює новий масив.
2. Новий масив може бути коротшим (або навіть порожнім `[]`), якщо не всі елементи пройшли перевірку.

```
const users = [
  { name: "Анна", age: 17 },
  { name: "Олег", age: 25 },
  { name: "Ігор", age: 15 },
];

// Залишаємо лише повнолітніх (age >= 18)
const adults = users.filter((user) => user.age >= 18);

console.log(adults); // Отримаємо масив з одним об'єктом: [{ name: "Олег", age: 25
}]
```

6. Агрегація даних (Згортання): `reduce`

Це найскладніший, але найбільш універсальний метод. `reduce` застосовується, коли потрібно "згорнути" (агрегувати) величезний масив в одне єдине фінальне значення (наприклад, знайти загальну суму всієї корзини покупок, знайти найдорожчий товар, або згрупувати масив об'єктів у словник).

Callback у `reduce` трохи відрізняється — він приймає 4 аргументи, але найважливішими є перші два:

1. `accumulator` (Акумулятор) — проміжний результат, який "накопичується" при проході по масиву. З кожним кроком у ньому зберігається те, що повернув Callback на *попередньому* кроці.
2. `currentValue` — поточний елемент масиву (як в `map` або `filter`).

Також метод `reduce` приймає другий загальний аргумент (після Callback'у) — **початкове значення акумулятора** (Initial Value).

```
const orders = [500, 1000, 2500];

// accumulator (acc) тут виступає як "загальна сума".
// Початкове значення (0) ми передали після коми в кінці методу.
```

```
const totalSum = orders.reduce((acc, currentVal) => {
  return acc + currentVal;
}, 0);

console.log(totalSum); // 4000
```

Як це працювало всередині:

- Крок 1: acc = 0, currentVal = 500. Повертаємо 500.
- Крок 2: acc = 500, currentVal = 1000. Повертаємо 1500.
- Крок 3: acc = 1500, currentVal = 2500. Фінал: 4000.

7. Пошук та Перевірки (`find` , `some` , `every`)

Метод `find` та `findIndex`

Шукає в масиві **перший** об'єкт, який відповідає умові. Якщо знаходить — одразу повертає сам об'єкт (а не масив) і перериває пошук. Якщо не знайдено нічого — повертає `undefined` . `findIndex` робить те саме, але повертає не сам об'єкт, а його індекс (порядковий номер) у масиві (або `-1` якщо не знайдено).

```
/* Приклад: знайти користувача з id === 3 */
const foundUser = dbUsers.find((user) => user.id === 3);
```

Методи `some` та `every`

Повертають лише `true` або `false` (Boolean). Корисні для перевірок дозвілів чи валідації.

- `some` : Перевіряє, чи є в масиві **ХОЧА Б ОДИН** елемент, що відповідає умові.
- `every` : Перевіряє, чи **АБСОЛЮТНО ВСІ** елементи відповідають умові.

```
const ages = [18, 20, 16, 21];

const hasMinors = ages.some((age) => age < 18); // true (бо є 16)
const allAdults = ages.every((age) => age >= 18); // false (бо є 16)
```

Метод `sort`

Використовується для сортування масивів (наприклад, сортування товарів за ціною з найменшої). **Обережно (Мутація):** На відміну від `map` або `filter`, метод `sort` **змінює ("мутує") оригінальний масив**! Якщо сортувати без Callback-функції, `sort` перетворює всі значення на рядки тексту і сортує їх за абеткою (тоді `10` буде стояти перед `2`). Для чисел потрібно передавати Callback `(a, b)`:

```
const nums = [40, 100, 1, 5, 25];

// Сортування за зростанням
nums.sort((a, b) => a - b); // [1, 5, 25, 40, 100]

// Сортування за спаданням
nums.sort((a, b) => b - a); // [100, 40, 25, 5, 1]
```

8. Ланцюжки методів (Chaining)

Оскільки `map` та `filter` завжди повертають нові масиви, ми можемо застосовувати їх один за одним у вигляді ланцюжка (Chaining). Кожен наступний метод застосовується до результату роботи попереднього. Точка ставиться відразу після закриваючої дужки попереднього методу.

Задача: Дано масив продуктів. Потрібно знайти всі товари, що дешевші за 50 грн, і отримати масив лише з їхніх назв (рядків), записаних Великими Літерами.

```
const products = [
  { name: "Хліб", price: 20 },
  { name: "Сир", price: 150 },
  { name: "Молоко", price: 35 },
];

const cheapProductNames = products
  .filter((product) => product.price < 50) // Крок 1: Залишаємо [Хліб, Молоко]
  .map((product) => product.name.toUpperCase()); // Крок 2: Трансформуємо у
["ХЛІБ", "МОЛОКО"]

console.log(cheapProductNames); // ["ХЛІБ", "МОЛОКО"]
```

Такий підхід дозволяє створювати надзвичайно потужні потоки обробки даних, уникаючи створення зайвих проміжних змінних (на кшталт `filteredArr`, `finalArr` тощо).

Висновки

1. Перехід від циклу `for` до методів масивів (`map` , `filter`) означає перехід від імперативного до декларативного стилю програмування (ваш код стає коротшим і зрозумілішим).
2. `forEach` є заміною `for` і використовується лише для того, щоб "щось зробити" з кожним елементом, але він нічого не повертає.
3. `map` створює та повертає новий масив тієї ж довжини (використовується для трансформації об'єктів або рендерингу).
4. `filter` створює новий масив, відкидаючи елементи, що не відповідають заданій умові.
5. `reduce` вміє згортати цілий масив у будь-яке єдине значення завдяки змінній `accumulator` .
6. `find` ідеально підходить для пошуку одного унікального об'єкта в базі за його `id` .
7. `sort` — небезпечний, оскільки мутує оригінальний масив і вимагає написання математичної callback-перевірки `(a, b) => a - b` .
8. Методи, що повертають масив (`map` , `filter` , `sort`), можна об'єднувати у довгі елегантні ланцюжки (Chaining).

Джерела

1. MDN Web Docs: Array - Офіційна документація по усім методам.
2. Javascript.info: Методи масивів - Сучасний підручник (Укр).
3. Відео-лекція: JavaScript Array Methods (English) - Огляд `map` , `filter` , `reduce` за 15 хвилин.

Запитання для самоперевірки

1. З якими аргументами викликається Callback-функція в методах масиву? Який з них є обов'язковим для використання?
2. В чому принципова різниця між `forEach` та `map` ? У яких випадках слід обрати кожен із них?
3. Що буде, якщо у Callback-функції для методу `map` ви забудете написати команду `return` ? Яким буде результат (масив з чим)?
4. Метод `filter` отримав Callback, який завжди повертає число `1` . Що буде у відфільтрованому масиві?
5. Для чого використовується значення `accumulator` у методі `reduce` , і де в синтаксисі вказується початкове значення цього акумулятора?

6. Якщо вам потрібно знайти користувача "Ігор" і пересвідчитися, що він є в базі, чому метод `find` буде набагато ефективнішим вибором (з точки зору швидкодії сервера), ніж метод `filter`?
7. У вас є масив чисел `[5, 1, 15]`. Яким буде результат виклику `arr.sort()` без переданої Callback-функції і чому?
8. Що таке Chaining (ланцюжок методів) і завдяки якій властивості методів `filter` або `map` він став можливим?